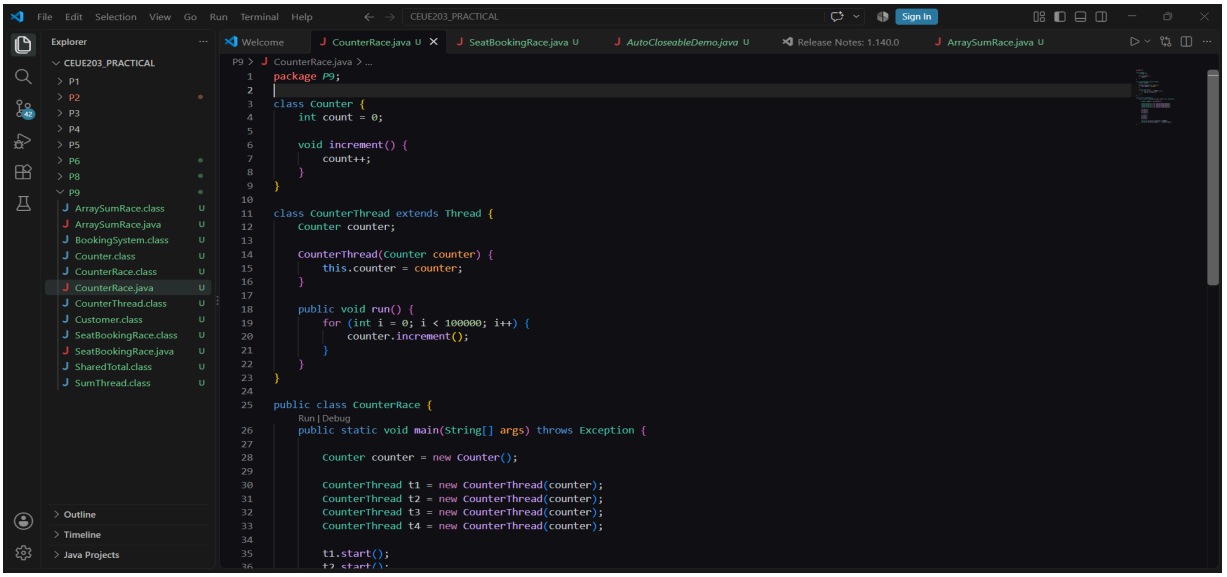
# PRACTICAL – 9

TASK 1 :

Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.

CODE :



```java
package P9;

class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}

class CounterThread extends Thread {
    Counter counter;

    CounterThread(Counter counter) {
        this.counter = counter;
    }

    public void run() {
        for (int i = 0; i < 100000; i++) {
            counter.increment();
        }
    }
}

public class CounterRace {
    Run | Debug
    public static void main(String[] args) throws Exception {

        Counter counter = new Counter();

        CounterThread t1 = new CounterThread(counter);
        CounterThread t2 = new CounterThread(counter);
        CounterThread t3 = new CounterThread(counter);
        CounterThread t4 = new CounterThread(counter);

        t1.start();
```



```java
public class CounterRace {
    Run | Debug
    public static void main(String[] args) throws Exception {

        Counter counter = new Counter();

        CounterThread t1 = new CounterThread(counter);
        CounterThread t2 = new CounterThread(counter);
        CounterThread t3 = new CounterThread(counter);
        CounterThread t4 = new CounterThread(counter);

        t1.start();
        t2.start();
        t3.start();
        t4.start();

        t1.join();
        t2.join();
        t3.join();
        t4.join();

        System.out.println(x: "Expected = 400000");
        System.out.println("Actual = " + counter.count);
```

OUTPUT :

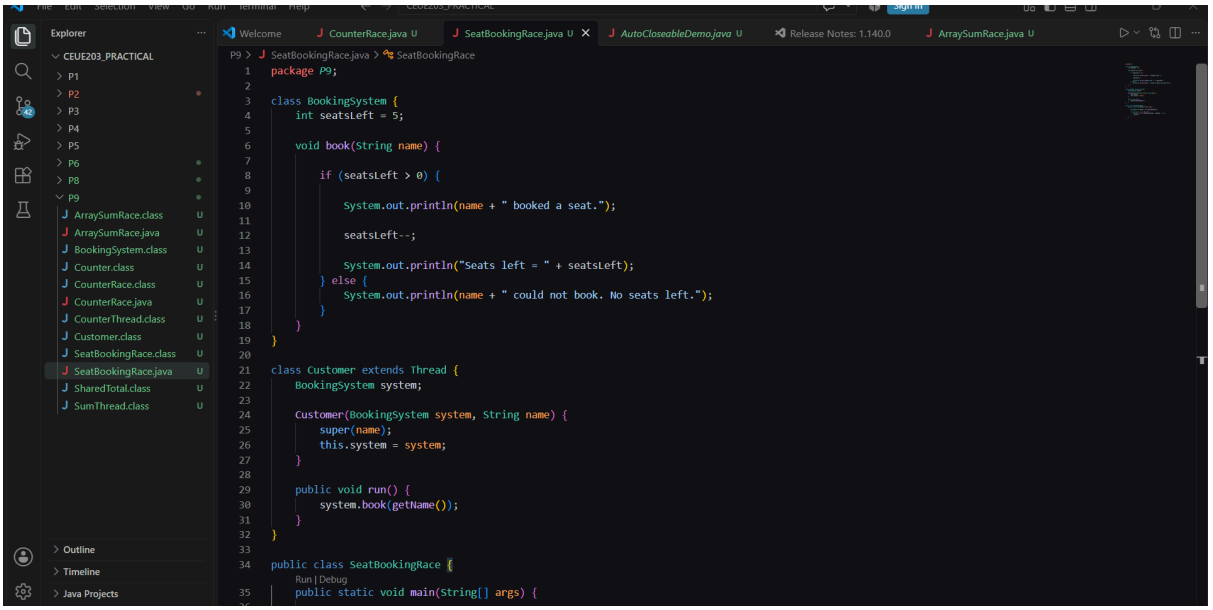


```
PS D:\CEUE203_PRACTICAL> & 'C:\Program Files\Java\jdk-21.0.12\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\krish\AppData\Roaming\Code\Us
er\workspaceStorage\bb52b99d8ee26e96980c9f492697c61b\redhat.java\jdt_ws\CEUE203_PRACTICAL_ae8cb560\bin' 'P9.CounterRace'
Expected = 400000
Actual = 150875
PS D:\CEUE203_PRACTICAL>
```

## TASK 2 :

Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with asynchronized book() exactly 5 do.

## CODE :



```java
package P9;

class BookingSystem {
    int seatsLeft = 5;

    void book(String name) {

        if (seatsLeft > 0) {

            System.out.println(name + " booked a seat.");

            seatsLeft--;

            System.out.println("Seats left = " + seatsLeft);
        } else {
            System.out.println(name + " could not book. No seats left.");
        }
    }
}

class Customer extends Thread {
    BookingSystem system;

    Customer(BookingSystem system, String name) {
        super(name);
        this.system = system;
    }

    public void run() {
        system.book(getName());
    }
}

public class SeatBookingRace {
    Run | Debug
    public static void main(String[] args) {
```



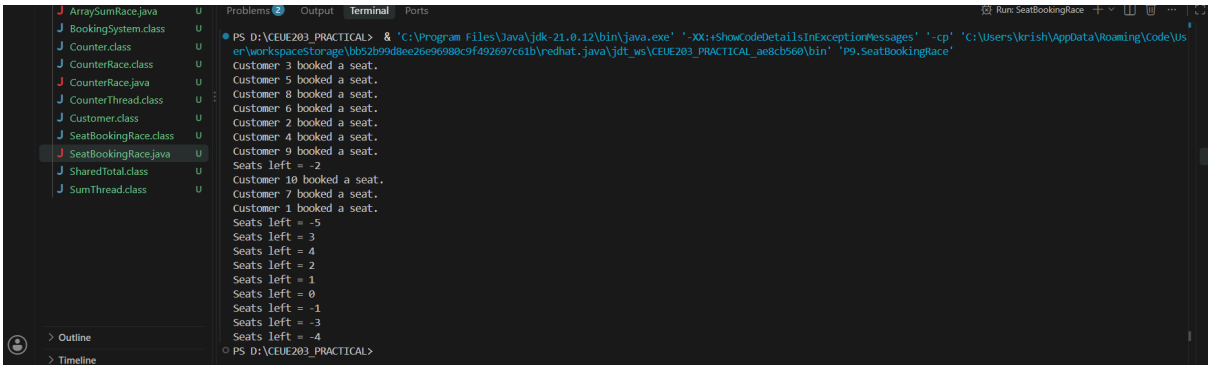
```java
    public void run() {
        system.book(getName());
    }
}

public class SeatBookingRace {
    Run | Debug
    public static void main(String[] args) {

        BookingSystem system = new BookingSystem();

        for (int i = 1; i <= 10; i++) {
            Customer c = new Customer(system, "Customer " + i);
            c.start();
        }
    }
}
```

## OUTPUT :

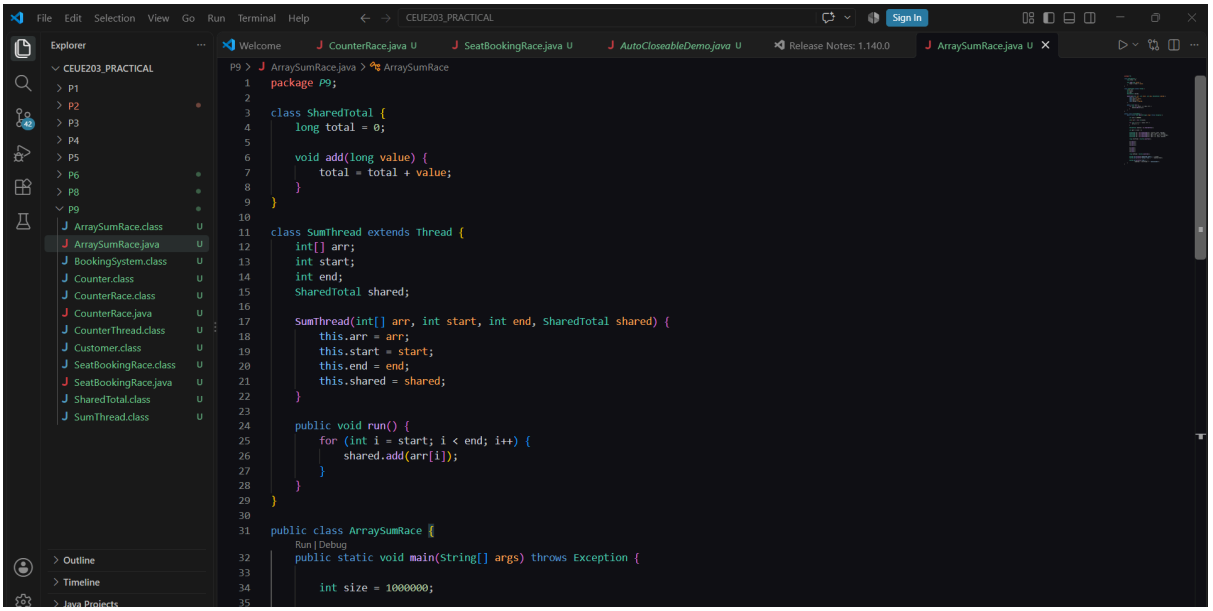


```
PS D:\CEUE203_PRACTICAL> & 'C:\Program Files\Java\jdk-21.0.12\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\krish\AppData\Roaming\Code\User\workspaceStorage\bb52b99d8ee26e96980c9f492697c61b\redhat.java\jdt_ws\CEUE203_PRACTICAL_ae8cb560\bin' 'P9.SeatBookingRace'
Customer 3 booked a seat.
Customer 5 booked a seat.
Customer 8 booked a seat.
Customer 6 booked a seat.
Customer 2 booked a seat.
Customer 4 booked a seat.
Customer 9 booked a seat.
Seats left = -2
Customer 10 booked a seat.
Customer 7 booked a seat.
Customer 1 booked a seat.
Seats left = -5
Seats left = 3
Seats left = 4
Seats left = 2
Seats left = 1
Seats left = 0
Seats left = -1
Seats left = -3
Seats left = -4
PS D:\CEUE203_PRACTICAL>
```

TASK 3 :

3. (For Advanced Learners – home) Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare;decide how to measure the difference.

CODE :



```java
package P9;

class SharedTotal {
    long total = 0;

    void add(long value) {
        total = total + value;
    }
}

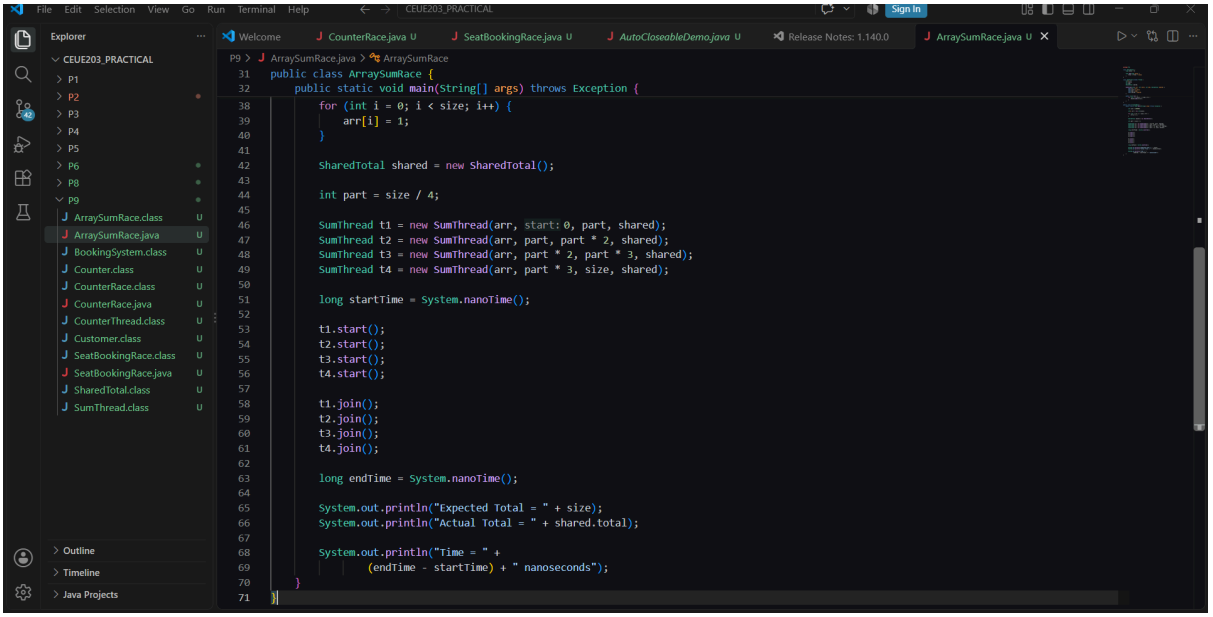
class SumThread extends Thread {
    int[] arr;
    int start;
    int end;
    SharedTotal shared;

    SumThread(int[] arr, int start, int end, SharedTotal shared) {
        this.arr = arr;
        this.start = start;
        this.end = end;
        this.shared = shared;
    }

    public void run() {
        for (int i = start; i < end; i++) {
            shared.add(arr[i]);
        }
    }
}

public class ArraySumRace {
    Run | Debug
    public static void main(String[] args) throws Exception {

        int size = 1000000;
```



```java
public class ArraySumRace {
    public static void main(String[] args) throws Exception {
        for (int i = 0; i < size; i++) {
            arr[i] = 1;
        }

        SharedTotal shared = new SharedTotal();

        int part = size / 4;

        SumThread t1 = new SumThread(arr, start: 0, part, shared);
        SumThread t2 = new SumThread(arr, part, part * 2, shared);
        SumThread t3 = new SumThread(arr, part * 2, part * 3, shared);
        SumThread t4 = new SumThread(arr, part * 3, size, shared);

        long startTime = System.nanoTime();

        t1.start();
        t2.start();
        t3.start();
        t4.start();

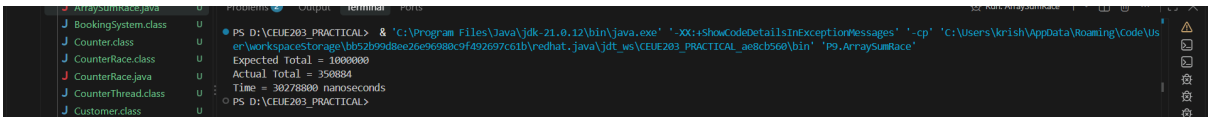
        t1.join();
        t2.join();
        t3.join();
        t4.join();

        long endTime = System.nanoTime();

        System.out.println("Expected Total = " + size);
        System.out.println("Actual Total = " + shared.total);

        System.out.println("Time = " +
                (endTime - startTime) + " nanoseconds");
    }
}
```

OUTPUT :




KEY QUESTIONS :

1. **What is a race condition, and why does it occur here?**
   A race condition happens when two or more threads access and change the same shared data at the same time. It occurs here because multiple threads are updating the counter or seats without proper synchronization, so the final result may become incorrect.
2. **What does the synchronized keyword do?**
   The synchronized keyword allows only one thread at a time to execute a particular method or block of code. It helps prevent race conditions and keeps shared data safe.
3. **What are the states in a thread's life cycle?**
   The main thread states in Java are: **NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED**.
   A thread starts in the NEW state, runs in RUNNABLE, may wait or get blocked during execution, and finally ends in the TERMINATED state.